EduDocAI: Technical Task Report

Track C: NLP / Generative AI — Candidate Task Assignment (EduRankAI)

Candidate: Kartikey Gupta | B.Tech CSE | k78048101@gmail.com

EduRankAI: Candidate Technical Task Report

Track C: NLP / Generative AI — Intelligent Document Intelligence System

Candidate Name: Kartikey Gupta
Role: AI Engineering & Data Science Intern
Project: EduRankAI (Track C: Intelligent Document Intelligence System)

---

1. Executive Summary & Problem Statement

1.1 Problem Statement

Modern enterprises and educational institutions manage vast repositories of unstructured textual information—policy handbooks, technical whitepapers, research articles, resumes, and support tickets. Extracting reliable, actionable answers from these documents is hampered by three major failure modes in standard conversational AI implementations:

1. **Hallucination & Fabricated Answers**: Large Language Models (LLMs) frequently extrapolate plausible but fictitious facts, numbers, dates, and citations when source evidence is incomplete.
2. **Cross-Document Contamination**: When an analyst queries a specific document (e.g. an employee policy or candidate profile), multi-document vector retrievers frequently return irrelevant fragments from other indexed documents, corrupting downstream reasoning.
3. **Operational Fragility**: In local execution environments (particularly Windows OS), standard file handlers often encounter file-locking permission crashes during temporary PDF handling.

1.2 Proposed Solution & Objectives

To solve these challenges, we designed and implemented **EduRankAI (Track C)**—a reproducible, production-style Document Intelligence system. EduRankAI processes documents in-memory via `io.BytesIO`, segments content using sentence-boundary sliding windows, isolates candidate search spaces via pre-retrieval filtering, benchmarks a **Dense Semantic Retrieval system (`all-MiniLM-L6-v2` + FAISS)** against a **Sparse TF-IDF baseline**, and validates all outputs through an automated grounding and citation gate.

1.3 Key Measured Outcomes

Across an empirical 22-question curated benchmark spanning 6 query categories:
- **Recall@3**: **100.00%** on both Dense and Sparse retrieval.
- **Mean Reciprocal Rank (MRR)**: **0.9318** (Dense) vs. **0.9242** (TF-IDF Baseline).
- **Citation Accuracy**: **90.91%** (Dense) vs. **81.82%** (TF-IDF).
- **Abstention Accuracy**: **90.91%** on unsupported queries.
- **Retrieval Latency**: **29.81 ms** (Dense) vs. **3.39 ms** (TF-IDF) on standard CPU.

---

2. Dataset Source, Licence, Data Dictionary, and Quality Audit

2.1 Dataset Provenance & Licensing

In strict accordance with EduRankAI's academic integrity rules forbidding personal/confidential data, the system was developed and benchmarked using a reproducible demonstration corpus of 3 authorized synthetic documents created specifically for this technical evaluation:
1. `TechCorp_Employee_Handbook_2025.pdf` / `.txt`: Multi-page human resources policy document detailing core collaboration hours, PTO accrual, healthcare coverage, and parental leave.
2. `EduDocAI_Architecture_Whitepaper.pdf` / `.txt`: Multi-page engineering whitepaper specifying system architecture, chunking parameters, FAISS vector indexing, and RRF formulas.
3. `Global_AI_Ethics_Guidelines_2025.pdf` / `.txt`: Research ethics document defining source provenance, abstention requirements, and ephemeral in-memory processing.
4. **User Document Support**: Fully tested with user-uploaded documents (e.g. `KARTIKEY_GUPTA.pdf`) in-memory without persistent cloud transmission.

Licence & Terms: Open Demonstration License (CC-BY 4.0 compatible), zero proprietary or confidential institutional data.

2.2 Data Dictionary

Field	Type	Description	Example
`doc_id`	String (Hex 12)	SHA-256 derived stable document identifier	`715563583354`
`source`	String	Original filename of the document	`TechCorp_Employee_Handbook_2025.pdf`
`page`	Integer	1-indexed document page number	`1`
`chunk_id`	String	Granular chunk locator: `{doc_id}_p{page}_c{idx}`	`715563583354_p1_c0`
`text` / `chunk_text`	String	Cleaned normalized textual proposition content	`"Full-time employees work 40 hours..."`
`word_count`	Integer	Word count of the indexed chunk	`128`

2.3 Data Quality Audit

- **Missing Values**: 0 nulls across extracted pages. Image-only pages or corrupt files are intercepted by custom exceptions (`EmptyDocumentError`, `CorruptedDocumentError`) with explicit OCR alerts.
- **Duplicates**: Deduplication performed at the SHA-256 chunk hash level to prevent indexing identical blocks.
- **Data Leakage Prevention**: All evaluation questions are held in a separate, isolated test set (`data/evaluation/eval_dataset.json`). No test set questions or ground-truth answers were used during TF-IDF vectorizer fitting or index generation.

---

3. Exploratory Analysis and Preprocessing Decisions

3.1 Diagnostic Visualizations (Work Package C)

Four diagnostic visualizations were generated and saved to `results/charts/`:
1. `01_document_sources.png`: Visualizes page count inventory across source files.

2. `02_chunk_length_distribution.png`: Demonstrates chunk length normality ($W=400$, $\sigma=68$, median$=385$ words), verifying that the sentence-boundary heuristic prevents truncating propositions.
3. `03_eval_question_categories.png`: Categorical breakdown of the 22 benchmark queries (Factual: 36%, Numeric: 23%, Paraphrased: 18%, Multi-Doc: 5%, Ambiguous: 5%, Abstention: 13%).
4. `04_dense_vs_tfidf_comparison.png`: Comprehensive grouped performance comparison across Recall@K, MRR, F1, and Abstention.

3.2 Preprocessing & Text Cleaning Decisions

- **NFKC Normalization**: Extracted raw strings undergo Unicode Compatibility Decomposition and Composition to standardize ligature variants.
- **Whitespace & Control Character Stripping**: Strips null bytes (`\x00`), non-breaking spaces (`\u00a0`), and zero-width spaces (`\u200b`, `\ufeff`).
- **Hyphen Repair**: Reconnects hyphenated line-broken words (e.g. `trans-\nformer` $\rightarrow$ `transformer`).
- **In-Memory Streaming**: File buffers are processed via `io.BytesIO` into PyMuPDF (`fitz`), eliminating Windows OS temporary file locks (`[WinError 32]`).

---

4. Baseline, Proposed Method, Architecture, and Implementation



4.1 Simple Baseline: Sparse TF-IDF Retriever

As mandated by Work Package D, a simple, interpretable baseline was built using Scikit-Learn's `TfidfVectorizer` with sublinear term-frequency scaling and unigram/bigram ranges `(1, 2)`. Query vectors are projected against document chunk vectors using cosine similarity:

$$\text{Sim}_{\text{tfidf}}(q, c) = \frac{\mathbf{w}_q \cdot \mathbf{w}_c}{|\mathbf{w}_q|_2 |\mathbf{w}_c|_2}$$

4.2 Proposed Model: Dense Semantic Retrieval (`all-MiniLM-L6-v2` + FAISS)

The primary retrieval system leverages dense semantic representations generated by `sentence-transformers/all-MiniLM-L6-v2`. Vectors $\mathbf{v} \in \mathbb{R}^{384}$ are L2-normalized:

$$\mathbf{v}c = \frac{\text{Embed}(c)}{|\text{Embed}(c)|_2}, \quad \mathbf{v}_q = \frac{\text{Embed}(q)}{|\text{Embed}(q)|_2}$$

Cosine similarity is computed via inner product on `faiss.IndexFlatIP`:

$$\text{Sim}_{\text{dense}}(q, c) = \mathbf{v}_q \cdot \mathbf{v}_c$$

4.3 Pre-Retrieval Document-Level Filtering

To strictly prevent cross-document contamination when querying specific files (e.g. `KARTIKEY_GUPTA.pdf` or `TechCorp_Employee_Handbook_2025.pdf`), filtering is applied **before** ranking. Candidate chunk indices $\mathcal{I}_{\mathcal{S}}$ are sliced prior to vector multiplication, guaranteeing zero leakage from unselected documents.

4.4 Grounding Gate & Extractive Fallback

When local LLM inference (Ollama) is offline, the system utilizes a deterministic extractive QA engine that ranks sentences by query-token overlap, numeric entity presence, and retrieval similarity. If evidence overlap $\Omega(A, \mathcal{E}) < 0.30$ or retrieval score falls below threshold $\tau=0.15$, the system enforces canonical abstention:

> *"The requested information was not found in the uploaded documents."*

---

5. Experimental Protocol, Metrics, Results, and Comparison

5.1 Evaluation Setup

- **Evaluation Set**: 22 curated test cases (`data/evaluation/eval_dataset.json`).
- **Metrics**: Recall@1, Recall@3, Recall@5, Mean Reciprocal Rank (MRR), Token F1 Score, Citation Accuracy, Abstention Accuracy, and Retrieval Latency.
- **Execution Script**: `python scripts/evaluate.py`.

5.2 Comparative Results Table

Evaluation Metric	Sparse Baseline (`TF-IDF`)	Proposed Dense (`all-MiniLM-L6-v2`)	Performance Advantage
Recall@1	0.8636 (86.36%)	**0.8636 (86.36%)**	Parity
Recall@3	1.0000 (100.0%)	**1.0000 (100.0%)**	Parity (100% coverage)
Recall@5	1.0000 (100.0%)	**1.0000 (100.0%)**	Parity
Mean Reciprocal Rank (MRR)	0.9242	**0.9318**	**+0.82% Dense advantage**
Token F1 Score	0.4563	**0.4763**	**+4.38% Dense advantage**
Citation Accuracy	0.8182 (81.82%)	**0.9091 (90.91%)**	**+11.11% Dense advantage**
Abstention Accuracy	0.8182 (81.82%)	**0.9091 (90.91%)**	**+11.11% Dense advantage**
Retrieval Latency (ms)	**3.39 ms**	29.81 ms	Baseline 8.8x faster
Total Response Latency (ms)	**4.17 ms**	30.83 ms	Both real-time on CPU

5.3 Analysis of Findings

1. **Semantic Variance**: On paraphrased questions (e.g. asking about "healthcare family coverage" when the text states "eligible dependents"), dense embeddings correctly ranked the target passage at Rank #1, whereas TF-

IDF required Rank #2 or #3 due to lexical mismatch.

2. **Abstention Superiority**: Dense semantic retrieval demonstrated tighter similarity calibration on out-of-domain queries (e.g. questions regarding stock grants or travel allowances), correctly abstaining on 90.91% of negative controls compared to 81.82% for TF-IDF.
3. **Latency Trade-Off**: TF-IDF provides an ultra-lightweight 3.39 ms retrieval speed, making it an ideal fallback on edge devices with constrained memory.

---

6. Error Analysis, Limitations, Ethics/Security, and Intended-Use Boundaries

6.1 Representative Successes, Failures, and Edge Cases

- **Success Case (Q03 - Numeric Precision)**: "*What is the annual home-office stipend?*" $\rightarrow$ Both models retrieved Page 1 with score 0.81, and the extractive engine isolated `"$1,200 annual stipend"` with 100% token accuracy.
- **Success Case (Q19 - Safe Abstention)**: "*What is the company policy on annual stock option grants?*" $\rightarrow$ Similarity was below safety threshold $\tau=0.15$; system safely emitted the canonical abstention message with zero hallucination.
- **Failure / Edge Case (Q18 - Broad Ambiguous Query)**: "*What is the leave policy?*" $\rightarrow$ The document contained both PTO and Parental Leave. Retrieval captured PTO (Page 1) at Rank #1 but pushed Parental Leave (Page 2) to Rank #2. In single-chunk answering mode, parental leave was omitted unless top-K was set to 3.

6.2 Limitations & Mitigations

1. **Scanned Documents**: The current pipeline processes digital text PDFs. Image-only scans trigger an informative message requiring OCR. *Mitigation*: Integrate `pytesseract` or `easyocr`.
2. **Complex Tables**: Multi-column ASCII tables may experience word wrap in linear extraction. *Mitigation*: Integrate layout-aware PDF parsers like `pdfplumber`.

6.3 Responsible AI & Security Boundaries

Privacy & Data Handling:
All user-uploaded documents are processed entirely in-memory using `io.BytesIO` streams. No document content is written to disk, logged, transmitted to any third-party cloud service, or retained between sessions. Each Streamlit session maintains its own isolated state — one user's documents are never visible to another session.

Intended Use & Scope:
EduDocAI is designed for informational document Q&A with human-in-the-loop verification. It is not certified or intended for standalone use in legal, medical, financial, or clinical decision-making. All answers should be verified against the cited source page before acting on them for high-stakes decisions. The system's abstention mechanism ("not found") is a safety feature, not a guarantee of completeness.

Fairness & Bias:
The system's answers are grounded in the content of the documents provided by the user. It does not introduce external knowledge or cultural bias from a pre-trained language model when running in extractive mode. In LLM mode (Ollama), the underlying model's training biases may influence phrasing — but factual claims are still gated by the grounding validator against the source text.

Security & Adversarial Robustness:
Ten adversarial inputs were tested (see `reports/attack_log.md`), including prompt injection, data exfiltration requests, jailbreak attempts via role assignment, and false-premise manipulation. All 10 passed. The extractive

pipeline is inherently resistant to prompt injection because the core answer engine is a deterministic scoring function, not an instruction-following language model.

Uncertainty & Limitations:
- The system does not process scanned (image-only) PDFs — requires OCR integration.
- Complex multi-column tables may extract incorrectly in linear text mode.
- The 22-question evaluation benchmark, while covering 6 categories, may not fully represent the diversity of real-world user queries.
- Performance on non-English documents has not been evaluated.

---

7. Conclusion and Prioritised Next Steps

7.1 Conclusion

The EduDocAI Track C implementation successfully demonstrates that a disciplined, well-evaluated RAG architecture with sentence-boundary chunking, document-level pre-retrieval filtering, and dual retrieval (Dense + TF-IDF) outperforms naive approaches. It achieves **100% Recall@3**, **0.9318 MRR**, and **90.91% citation/abstention accuracy** with sub-35ms CPU latency — running fully offline without any paid API.

Live system: https://edudocai.streamlit.app/
Source code: https://github.com/guptaKartikey/EduDocAI

7.2 Prioritised Next Steps

1. **OCR Integration**: Add `pytesseract` or `easyocr` to support scanned/image PDFs, which currently cannot be processed.
2. **Layout-Aware Extraction**: Integrate `pdfplumber` for structured table and multi-column extraction to improve accuracy on financial and academic documents.
3. **Cross-Encoder Reranking**: Add `ms-marco-MiniLM-L-6-v2` cross-encoder for improved multi-hop and paraphrase-heavy queries.
4. **Persistent Index Storage**: Support ChromaDB or on-disk FAISS serialization to avoid re-indexing large corpora on every app restart.
5. **Embedding-Based Grounding**: Replace token-overlap grounding with semantic embedding similarity for more accurate paraphrase-aware validation.

---

8. Daily Build & Execution Log (3-Day Build Assessment)

As required by the candidate brief, the end-to-end development was tracked through daily structured check-ins documenting completed milestones, technical blockers, hours invested, and engineering decisions.

8.1 Day 1: Knowledge Ingestion, Baselines, and Measurement

- **Milestones Completed**:
- Established modular project architecture: ingestion, sentence-aware chunking, vector indexing, and retrieval.
- Built and benchmarked the Sparse TF-IDF baseline retriever as the benchmark anchor (Recall@3 = 100%, MRR = 0.9242) across 22 annotated ground-truth test cases.
- Discovered and resolved Windows OS file-locking permissions (`[WinError 32]`) by transitioning from disk writes to pure in-memory `io.BytesIO` streams.

- Completed dataset provenance audit, data dictionary, and leakage-prevention protocols.
- **Key Challenge & Recovery**: PyMuPDF failed on temporary file access under Windows file locks. Re-architected ingestion to pass byte streams directly to `fitz.open(stream=..., filetype="pdf")`, completely eliminating disk I/O.
- **Time Invested**: ~5.5 focused hours.

8.2 Day 2: Semantic Retrieval, Grounding Gate, and Adversarial Hardening

- **Milestones Completed**:
- Implemented Dense Semantic Retrieval using `sentence-transformers/all-MiniLM-L6-v2` and FAISS inner-product vector search.
- Developed pre-retrieval candidate masking to prevent cross-document contamination during multi-file indexing.
- Built automated Grounding & Citation Validator enforcing token-level evidence verification and zero-hallucination abstention.
- Profiling and resolving evaluation latency: eliminated a 4-second socket probe delay on offline Ollama checks via a 30-second TTL availability cache (reducing total eval time from 66s to 2.61s).
- Executed 10 adversarial attacks (prompt injection, exfiltration, persona jailbreak) with 100% safety pass rate.
- **Key Challenge & Recovery**: Broad summarization queries initially triggered false-positive abstentions due to strict lexical overlap thresholds. Solved by implementing an intent-aware classifier with adaptive grounding thresholds for summary requests.
- **Time Invested**: ~6.0 focused hours.

8.3 Day 3: Usability, Packaging, and Production Readiness

- **Milestones Completed**:
- Streamlined Streamlit application into a clean 3-page production workflow (Home & Upload, Chat with Documents, Evaluation Dashboard).
- Built 1-click Benchmark Sample Document loader (`TechCorp`, `AI Ethics`, `Whitepaper`) for instant reviewer reproducibility.
- Created automated regression test guard (`scripts/regression_check.py`) enforcing metric quality gates.
- Embedded unified brand visual identity (logo, typography, clean contrast footer) across all application pages.
- Authored comprehensive Failure & Recovery Log (`FAILURES.md`) and operational founder memo (`reports/decision_memo.md`).
- Deployed live production app to Streamlit Community Cloud and synchronized the complete repository on GitHub.
- **Time Invested**: ~5.0 focused hours.

---

9. Contribution Statement

This project represents my own original work. The system architecture, chunking strategy, retrieval pipeline design, grounding validation logic, evaluation dataset curation, and written analysis are my own engineering and analytical decisions.

AI tools used:
- **Google Gemini / Antigravity (AI coding assistant):** Used to accelerate boilerplate code generation (Streamlit UI layout, pytest test scaffolding, CSS styling). All generated code was reviewed, tested, and understood before

inclusion. I can explain and modify any line of this codebase.
- **No AI-generated text was used in the evaluation results, metric calculations, or experimental conclusions** — all numbers are computed by the actual pipeline on the actual benchmark data.

Third-party libraries and components:
- `sentence-transformers` (Apache 2.0) — embedding model
- `faiss-cpu` (MIT) — vector index
- `scikit-learn` (BSD-3) — TF-IDF vectorizer
- `PyMuPDF / fitz` (AGPL-3.0) — PDF extraction
- `Streamlit` (Apache 2.0) — UI framework
- `pytest` (MIT) — testing framework

Data: All benchmark documents (`data/sample_documents/`) were created specifically for this evaluation under an open demonstration license (CC-BY 4.0 compatible). No confidential, personal, or institutional data was used.

Declaration: I confirm that the submitted work accurately represents my contribution. I have identified all external sources and disclosed AI tools used. I understand I may be asked to explain or reproduce any part of this implementation.

EduDocAI — AI Engineering & Data Science Technical Assessment | Kartikey Gupta | All Rights Reserved © 2026